A PROJECT REPORT ON

CIPHER: AI-POWERED PROCTORED CODING EXAM PLATFORM

Submitted in fulfillment of the requirements for the
Award of the degree of

B. TECH COMPUTER SCIENCE AND DESIGN

T.E. Semester VI

Engineering Design and Innovation IV

SUBMITTED BY

1) Shantanu Sharma
Exam No: BTCSD6050

2) Atharva Kesare
Exam No: BTCSD6020

3) Ishankit Patil
Exam No: BTCSD6038

4) Priyanshu Raj
Exam No: BTCSD6041

Under the Guidance of

Prof. Priyanka Pawar

DR. D.Y. PATIL SCHOOL OF SCIENCE AND TECHNOLOGY, TATHAWADE, PUNE.

2025 – 2026

DR. D. Y. PATIL VIDYAPEETH, PUNE

Dr. D. Y. Patil School of Science & Technology, Tathawade
Department of Computer Science and Design

CERTIFICATE

This is to certify that the Project report entitled

"Cipher: AI-Powered Proctored Coding Exam Platform"

submitted by

1) Shantanu Sharma	Exam No: BTCSD6050
2) Atharva Kesare	Exam No: BTCSD6020
3) Ishankit Patil	Exam No: BTCSD6038
4) Priyanshu Raj	Exam No: BTCSD6041

is a Bonafide work carried out by the above students of this institute under the supervision of **Prof. Priyanka Pawar** and it is approved for partial fulfillment of the requirement of Dr. D.Y. Patil Vidyapeeth for the award of the degree B.TECH in Computer Science and Design (T.E. Semester VI, Engineering Design and Innovation IV).

Project Guide **Head of the Department**

ACKNOWLEDGEMENTS

It is our privilege to express our sincerest regards to our project guide, **Prof. Priyanka Pawar**, for her valuable inputs, able guidance, encouragement, whole-hearted cooperation, and constructive criticism throughout the duration of our project.

We deeply express our sincere thanks to our Head of Department **Mrs. Priyanshi Mulwani** for encouraging and allowing us to present the project on the topic "Cipher: AI-Powered Proctored Coding Exam Platform" at our department premises.

We take this opportunity to thank all our faculties who directly or indirectly helped our project. We pay our respect and love to our parents and all other family members and friends for their love and encouragement throughout our career. Last but not the least we express our thanks to our friends for their cooperation and support.

ABSTRACT

The exponential growth of remote education has introduced significant challenges in maintaining academic integrity during online coding examinations. Traditional proctoring methods are either too expensive, overly invasive, or easily circumvented by tech-savvy students.

Cipher is an AI-powered proctored coding exam platform that addresses these challenges by combining client-side artificial intelligence for behavioral monitoring with sandboxed code execution in Docker containers. The platform performs real-time face detection and identity verification using TensorFlow.js-based face-api.js models running entirely in the student's browser, ensuring zero data leaves the device and preserving privacy.

The system enforces a multi-layered anti-cheat framework including mandatory fullscreen mode with hard-blocking overlays, tab-switch and window-blur detection, clipboard interception, phone detection via COCO-SSD object detection in isolated Web Workers, and browser navigation trapping. All proctoring events are transmitted in real-time to instructors via WebSocket channels.

Student code is executed in ephemeral Docker containers with strict resource limits (256MB RAM, 1 CPU, 3-second timeout) and zero network access, supporting Python, JavaScript, and SQL. The platform features role-based dashboards for students, teachers, and administrators, with capabilities for exam creation, live monitoring, manual grading, and plagiarism detection.

Built on a three-tier architecture using Next.js 16, Node.js/Express, and MongoDB Atlas, Cipher demonstrates that secure, privacy-first proctoring can be achieved without external paid APIs or server-side video processing.

Keywords: AI proctoring, online examination, face detection, sandboxed code execution, academic integrity, Docker, WebSocket, TensorFlow.js

TABLE OF CONTENTS

Acknowledgement iii

Abstract iv

List of Figures vi

List of Tables vii

List of Acronyms viii

1. INTRODUCTION 1

1.1 Overview 1

1.2 Motivation 1

1.3 Problem Statement 2

1.4 Objectives 2

2. LITERATURE SURVEY 3

3. REQUIREMENT SPECIFICATIONS 5

3.1 Hardware Requirements 5

3.2 Software Requirements 5

3.3 Functional Requirements 6

3.4 Non-Functional Requirements 6

4. SYSTEM DESIGN 7

4.1 System Architecture 7

4.2 UML Diagrams 8

5. IMPLEMENTATION DETAILS 10

5.1 Description of Tools 10

5.2 Programming Language Description 11

5.3 Algorithm Details 11

5.4 Output of Implementation (Screenshots) 12

6. TESTING 15

6.1 Types of Testing 15

7. RESULTS AND CONCLUSION 17

7.1 Conclusion 17

7.2 Future Scope 17

References 18

LIST OF FIGURES

Fig 4.1 .....Three-Tier System Architecture 7

Fig 4.2 .....Use Case Diagram 8

Fig 4.3 .....State Chart – Exam Session Lifecycle 9

Fig 4.4 .....Activity Diagram – Proctoring Flow 9

Fig 5.1 .....Landing Page 12

Fig 5.2 .....Login Page with Face ID 12

Fig 5.3 .....Teacher Dashboard 13

Fig 5.4 .....Exam Creation Interface 13

Fig 5.5 .....Student Exam Environment 14

Fig 5.6 .....Live Proctoring Monitor 14

LIST OF TABLES

Table 2.1 .....Literature Survey Summary 3

Table 3.1 .....Functional Requirements 6

Table 5.1 .....Technology Stack 10

Table 6.1 .....Test Cases 15

LIST OF ACRONYMS

Acronym	**Full Form**
AI	Artificial Intelligence
API	Application Programming Interface
JWT	JSON Web Token
REST	Representational State Transfer
SSR	Server-Side Rendering
UML	Unified Modeling Language
CRUD	Create, Read, Update, Delete
SQL	Structured Query Language
CSS	Cascading Style Sheets
DOM	Document Object Model

1. INTRODUCTION

1.1 Overview

Cipher is an AI-powered proctored coding exam platform designed for educational institutions to conduct secure online programming examinations with integrated anti-cheating measures. The platform combines browser-based artificial intelligence for real-time behavioral monitoring with sandboxed code execution in Docker containers, creating a tamper-proof environment for technical assessments.

The system serves three user roles — Students, Teachers, and Administrators — each with dedicated dashboards. Teachers can create exams with configurable duration and warning limits, add coding questions across multiple languages (Python, JavaScript, SQL), and monitor live exam sessions. Students take exams in a secured full-screen environment with real-time AI proctoring, write and execute code in a professional CodeMirror editor, and receive instant feedback.

1.2 Motivation

The rapid shift to online education has created an urgent need for reliable remote examination solutions. Existing online proctoring systems suffer from several critical limitations:

- **Privacy concerns:** Most solutions record and stream video to external servers, raising data privacy issues.
- **High cost:** Commercial proctoring services charge per-student fees that are prohibitive for many institutions.
- **Easy circumvention:** Simple tab-switching, screen-sharing, or using a secondary device can bypass basic proctoring.
- **No code execution:** General-purpose proctoring tools lack integrated coding environments with safe execution capabilities.

Cipher was motivated by the need for a purpose-built, privacy-first, cost-effective solution that combines AI proctoring specifically with coding examination capabilities.

1.3 Problem Statement

To design and develop a secure, web-based coding examination platform that employs client-side AI for real-time behavioral monitoring and identity verification while executing student code in isolated, resource-constrained Docker containers — ensuring academic integrity without compromising user privacy or requiring expensive external services.

1.4 Objectives

1. Develop a secure coding exam environment with mandatory fullscreen enforcement and anti-cheat mechanisms.
2. Implement client-side AI face detection, identity verification, and phone detection using TensorFlow.js.
3. Build a sandboxed code execution engine supporting Python, JavaScript, and SQL with strict resource limits.
4. Create real-time WebSocket-based monitoring dashboards for instructors.
5. Design a role-based system with dedicated interfaces for students, teachers, and administrators.
6. Ensure zero external data transmission for proctoring — all AI runs locally in the browser.

2. LITERATURE SURVEY

A comprehensive review of existing research and systems was conducted to understand the current landscape of online proctoring and secure code execution platforms.

Table 2.1: Literature Survey Summary

Sr.	Reference	Technology	Task	Key Result
1	Proctorio (2020) - AI Proctoring Tool	Browser Extension, ML	Automated remote proctoring	Effective but raises privacy concerns due to server-side video analysis
2	HackerRank CodePair	WebSocket, Docker	Live coding interviews	Real-time code execution but lacks AI proctoring capabilities
3	Agarwal et al. (2021) - Face Recognition for Exams	OpenCV, LBPH	Identity verification in exams	92% accuracy in controlled lighting; struggled with varied environments
4	LeetCode Contest System	React, Kubernetes	Competitive coding platform	Scalable execution but no proctoring or anti-cheat enforcement
5	Kaur et al. (2022) - Browser-Based Proctoring	JavaScript, WebRTC	Tab-switch and focus detection	Reduced cheating by 40% with browser-level monitoring
6	Judge0 - Open Source Code Executor	Docker, REST API	Sandboxed code execution	Supports 60+ languages; resource isolation via cgroups
7	face-api.js (Justadudewhohacks)	TensorFlow.js, CNN	Client-side face detection	Real-time face landmark detection at 30fps in browser
8	Singh et al. (2023) - AI Cheating Detection	YOLO, OpenCV	Object detection for phones	95% phone detection accuracy but requires server GPU
9	Examity Online Proctoring	AI + Human Review	Hybrid proctoring	High accuracy but expensive per-session cost model
10	COCO-SSD (TensorFlow)	MobileNet, SSD	Real-time object detection	Detects 80 object classes including phones; runs in-browser via TF.js

Key Insights from Literature Survey:

- Most proctoring solutions rely on server-side video processing, creating privacy and cost concerns.
- Client-side AI using TensorFlow.js enables privacy-preserving face detection without video uploads.
- Docker-based sandboxing is the industry standard for secure code execution with resource isolation.
- No existing solution combines browser-based AI proctoring with integrated coding exam capabilities.
- Cipher fills this gap by unifying client-side AI proctoring with Docker-sandboxed code execution.

3. REQUIREMENT SPECIFICATIONS

3.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 / AMD Ryzen 3 or equivalent
RAM	4 GB (8 GB recommended)
Storage	500 MB free disk space
Webcam	720p webcam for face detection
Network	Stable internet connection (5 Mbps+)
Display	1280×720 minimum resolution

3.2 Software Requirements

Software	Version	Purpose
Node.js	v22+	Backend runtime & SQL execution
Next.js	16.1.6	Frontend React framework
MongoDB Atlas	Cloud	NoSQL database
Docker Desktop	Latest	Sandboxed code execution
Redis (Upstash)	Cloud	Job queue for BullMQ
Modern Browser	Chrome/Edge/Firefox	Client application
Python	3.9+	Code execution support

3.3 Functional Requirements

Table 3.1: Functional Requirements

FR-ID	Requirement	Priority
FR-01	User Registration & Login with JWT authentication	High
FR-02	Face registration and Face ID login via face-api.js	High
FR-03	Exam creation with configurable duration, questions, and warning limits	High
FR-04	Multi-language code editor (Python, JavaScript, SQL) with syntax highlighting	High
FR-05	Fullscreen enforcement with hard-blocking violation overlay	High
FR-06	Tab-switch, window-blur, and clipboard interception detection	High
FR-07	AI face detection, gaze tracking, and phone detection	High
FR-08	Secure code execution in Docker containers with resource limits	High
FR-09	Real-time instructor monitoring dashboard via WebSocket	Medium
FR-10	Manual grading interface for teachers with submission review	Medium

3.4 Non-Functional Requirements

- **Performance:** Handle 100 concurrent exam sessions with response time < 2 seconds.
- **Security:** HTTPS communication, bcrypt password hashing, JWT authentication, sandboxed execution.
- **Privacy:** All AI proctoring runs client-side; no video transmitted to server.

- **Availability:** 99.5% uptime during exam periods.
- **Scalability:** Three-tier architecture supporting horizontal scaling of each layer independently.